COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S2
DATE PERFORMED:	08/05/2026	**DATE SUBMITTED:**	08/05/2026
NAME:	Juan Paulo C. Lara	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 5.1 Queues

1. PROCEDURE OUTPUT

What procedures did you perform?

ILO A

```cpp
#include <iostream>
#include <queue>

void display(std::queue<char> copyQ);

int main(){

    // create an object
    std::queue<char> myQ;

    // use the enqueue operation
    myQ.push('J');
    myQ.push('P');
    myQ.push('L');
    std::cout << "current back is: " << myQ.front() << std::endl;
    std::cout << "current back is: " << myQ.back() << std::endl;
    display(myQ);

    // use the dequeue operation
    myQ.pop();
    display(myQ);
    myQ.pop();
    display(myQ);
    myQ.pop();

    // check the queue if it is empty
    std::cout << "Is the queue empty?" << myQ.empty() << std::endl;
    return 0;
}
```

```cpp
// note only use the member functions of the queue STL
void display(std::queue<char> copyQ){
    // create a copy of the queue
    std::queue<char> temp = copyQ;

    // loop until empty
    while(!temp.empty()){
        // display the front
        std::cout << temp.front();
        // dequeue the front
        temp.pop();
    }
    // add a new line
    std::cout << std::endl;
}

```

Figure 1. STL Implementation of Queues.

This driver code implements queues using the STL library to perform a basic queue operation of pushing and popping characters in a queue list. The program also features an additional display feature to print the queue to the output console, which the STL does not include in its library. Enqueue and Dequeue operations are performed in the stack library instead of including a custom header file, which the other figures will demonstrate later on.

```
current front is: J
current back is: L
JPL
PL
L
Is the queue empty? 1
```

Figure 2. Output of the STL Implementation of Queues.

This is the output of the basic implementation of queues with the display function to print the elements inside the queue list. The characters J, P, and L are pushed starting from J and ending with L. After pushing, the queue shows that J is at the front of the queue and L at the back. This is because L is the last character to be inserted into the queue. After this, the whole set of characters in the queue are displayed and then dequeued individually to demonstrate the dequeue function. Then, the program checks if the queue is empty after all the elements are cleared.

ILO B1

```cpp
#ifndef QUEUELL_H
#define QUEUELL_H

// create a class node
template <typename T>
class qNode{
    public:
        T data;
        qNode* next;
};
// new node creator
template <typename T>
qNode<T>* new_node(T newData){
    // allocate a space for the new node
    qNode<T>* newNode = new qNode<T>;
    // store the newData to the newNode
    newNode -> data = newData;
    // point the newNode next to null
    newNode -> next = nullptr;

    return newNode;
```

```cpp
// enqueue
template <typename T>
void enqueue(T newData, qNode<T>** frontPtr, qNode<T>** backPtr){
    // create a new node
    qNode<T>* newNode = new_node(newData);
    // check if the queue is empty
    if((*frontPtr) == nullptr && (*backPtr) == nullptr){
        (*frontPtr) = newNode;
        (*backPtr) = newNode;
        return;
    }
    // if not then inser at the back
    (*backPtr)->next = newNode; // LINKS THE CURRENT BACK TO THE NEW NODE
    (*backPtr) = newNode; // ASSIGNS THE BACKPTR TO THE NEW NODE
}
```

```cpp
// dequeue
template <typename T>
void dequeue(qNode<T>** frontPtr, qNode<T>** backPtr){
    // check if the queue is empty
    if((*frontPtr) == nullptr && (*backPtr) == nullptr){
        std::cout << "The queue is empty" << std::endl;
        return;
    }
    // create a temporary variable to store the node to be deleted
    qNode<T>* deleteTemp = nullptr;
    // check if the queue has only 1 item
    deleteTemp = (*frontPtr);

    // assign the current front to deleteTemp
    if((*frontPtr)->next == nullptr && (*backPtr)->next == nullptr){
        // return only 1 item is inside the queue
        (*frontPtr) = nullptr;
        (*backPtr) = nullptr;
        delete deleteTemp;
        return;
    }

    (*frontPtr) = (*frontPtr)->next;
    deleteTemp->next = nullptr;

    // dekete the node
    delete deleteTemp;
}
```

```
68  // display all
69  template <typename T>
70  void display(qNode<T>** frontPtr, qNode<T>** backPtr) {
71      qNode<T>* temp = *frontPtr;
72
73      while(temp != nullptr){
74          std::cout << temp->data << " ";
75          temp = temp->next;
76      }
77
78      std::cout<<"\n";
79  }
80
81  // isEmpty
82  template <typename T>
83  bool isEmpty(qNode<T>* frontPtr, qNode<T>* backPtr){
84      if(frontPtr == nullptr && backPtr == nullptr){
85          std::cout << "The queue is empty" << std::endl;
86          return true;
87      }
88      return false;
89  }
90  #endif
```

Figure 1. Header File Containing The Linked List Implementation of Queues.

This header file contains linked list implementation of queues, which contains the standard structure of a linked list program. The structure is composed of a class node, new node, isEmpty, and the basic queue and enqueue operations. The class node declares the basic data type and pointer for the next variable. The new node performs allocation, stores data to a new node, and points the next node to null. Then, the basic queue operations, which are enqueue and dequeue, both have an if statement for the front and back pointer to check if the queue is empty, declare a new node and temporary variable, and assign the front to another temporary variable. Similar to the previous codes, an isEmpty function checks if the queue is empty, and a display function prints the currently available elements in the queue.

```cpp
#include <iostream>
#include "queuell.h"

int main(){
    qNode<char>* front = nullptr;
    qNode<char>* back = nullptr;

    enqueue('J', &front, &back);
    std::cout << "Front: " << front->data << " " << "Back: " << back->data << std::endl;

    enqueue('I', &front, &back);
    std::cout << "Front: " << front->data << " " << "Back: " << back->data << std::endl;

    enqueue('M', &front, &back);
    std::cout << "Front: " << front->data << " " << "Back: " << back->data << std::endl;

    std::cout << "All elements in the queue: \n";
    display(&front, &back);

    std::cout << "Testing the dequeue operator: \n";
    dequeue(&front, &back);
    std::cout << "Front: " << front->data << " " << "Back: " << back->data << std::endl;
    dequeue(&front, &back);
    std::cout << "Front: " << front->data << " " << "Back: " << back->data << std::endl;
    dequeue(&front, &back);

    std::cout << "All elements in the queue: \n";
    display(&front, &back);

    std::cout << "Test if the Queue is Empty \n";
    isEmpty(front, back);

    return 0;
}
```

Figure 2. Driver Code of the Linked List Implementation of Queues.

This driver code contains the basic queue and dequeue operations in the linked list implementation of queues, which imports the library from the header file of the previous figure. The front and back variables are set to the default null pointer to set the variables to zero in memory. The enqueue function takes the three letters J, I, and M to place them into the queue data, and the dequeue function deletes each element individually. The display function prints the elements of the list sequentially.

```
Front: J Back: J
Front: J Back: I
Front: J Back: M
All elements in the queue:
J I M
Testing the dequeue operator:
Front: I Back: M
Front: M Back: M
All elements in the queue:

Test if the Queue is Empty
The queue is empty
```

Figure 3. Output of the Linked List Implementation.

This console output contains the operations performed by the driver code using the functions in the header file. As seen in the first three lines, there are the three characters inserted into the queue one by one, which shows the J being the front and back at first, the J and I in the next enqueue, and the J and M in the last queue. Then,

all the elements in the queue are printed to the console using the display function. Testing the dequeue operator, each of the three letters are removed one by one by each function invocation until all the elements are dequeued. After this, a test to check if the queue is empty is executed, which calls the empty function to check if the queue is empty and display the resulting message.

ILO B2

```cpp
#include <iostream>
#include "queueArr.h"

int main(){

    // creating an object
    queueArr<int> q(5);

    // enqueue 10, 20, 30, 40, 50
    q.Enqueue(1);

    std::cout << "Front: " << q.Front() << std::endl;
    std::cout << "Back: " << q.Back() << std::endl;
    std::cout << "Size: " << q.Size() << std::endl;

    q.Enqueue(2);

    std::cout << "Front: " << q.Front() << std::endl;
    std::cout << "Back: " << q.Back() << std::endl;
    std::cout << "Size: " << q.Size() << std::endl;

    q.Enqueue(3);
    q.Enqueue(4);

    std::cout << "Front: " << q.Front() << std::endl;
    std::cout << "Back: " << q.Back() << std::endl;
    std::cout << "Size: " << q.Size() << std::endl;

    q.Enqueue(5);
    std::cout << "Front: " << q.Front() << std::endl;
    std::cout << "Back: " << q.Back() << std::endl;
    std::cout << "Size: " << q.Size() << std::endl;
    q.Enqueue(6);

    std::cout << "\nRemoved: " << q.Dequeue() << std::endl;
    std::cout << "Removed: " << q.Dequeue() << std::endl;

    q.Clear();
    std::cout << "Is the queue empty? " << (q.Empty() ? "yes" : "no") << std::endl;
    return 0;
}
```

Figure 1. Driver Code of the Array Implementation of Queues.

This driver code performs the basic queue operations using the header file containing the array implementation of queues, which contains the enqueue, dequeue, and clear functions. What differs from the previous implementation is the creation of an object to allocate the number of elements to be placed in an array. This queue contains the numbers 1, 2, 3, 4, 5, and 6 individually enqueued in the array. For each enqueuement, the

added values are printed for the front, back, and the size. The member variables call for the functions in the header file to return the values assigned to them. The clear function removes all the elements and the empty function verifies that the queue is now empty.

```
#ifndef QUEUEARR_H
#define QUEUEARR_H
#include <iostream>
// Array-based circular queue
template <typename T>
class queueArr{
    private:
        // pointer to dynamically allocate an array
        T* q_array;
        // maximum
        size_t q_capacity;
        // current number of elements in the queue
        size_t q_size;
        // index of the front element
        int q_front;
        // index of the back element
        int q_back;
    public:
        // constructor
        queueArr(size_t capacity = 10);

        // copy constructor
        queueArr(const queueArr& other);

        // copy assignment operator
        queueArr& operator = (const queueArr& other);

        // destructor
        ~queueArr();

        // queue operations:
        bool Empty();
        bool Full();
        size_t Size();
        void Clear();
        T Front();
        T Back();
        void Enqueue(T value);
        T Dequeue();

};
```

```cpp
// constructor
template <typename T>
queueArr<T>::queueArr(size_t capacity){
    // capacity, q_size, q_front, q_back and q_array
    q_capacity = capacity;
    q_size = 0;
    q_front = 0;
    q_back = -1;
    q_array = new T[q_capacity];

}

// *copy constructor
template <typename T>
queueArr<T>::queueArr(const queueArr& other){
    q_capacity = other.q_capacity;
    q_size = other.q_size;
    q_front = other.q_front;
    q_back = other.q_back;

    q_array = new T[q_capacity];

    for(size_t i = 0; i < q_capacity; i++){
        q_array[i] = other.q_array[i];
    }
}

```

```cpp
// copy assignment operator
template <typename T>
queueArr<T>& queueArr<T>::operator=(const queueArr& other){
    if(this != &other){
        delete[] q_array;

        q_capacity = other.q_capacity;
        q_size = other.q_size;
        q_front = other.q_front;
        q_back = other.q_back;

        q_array = new T[q_capacity];

        for(size_t i = 0; i < q_capacity; i++){
            q_array[i] = other.q_array[i];
        }
    }

    return *this;
}

// destructor
template <typename T>
queueArr<T>::~queueArr(){
    delete[] q_array;
}
```

```cpp
// Empty()
template <typename T>
bool queueArr<T>::Empty(){
    // check if the size is 0
    return q_size == 0;
}

// Full()
template <typename T>
bool queueArr<T>::Full(){
    // check if the q_size has reached the maximum capacity
    return q_size == q_capacity;
}

// find the current size at the array
template <typename T>
size_t queueArr<T>::Size(){
    return q_size;
}
```

```cpp
// clear
template <typename T>
T queueArr<T>::Front(){
    // check if the queue is empty
    if(Empty()){
        std::cout << "the queue is empty\n";
        return T();
    }
    // return the front
    return q_array[q_front];
}

// back
template <typename T>
T queueArr<T>::Back(){
    // check if the queue is empty
    if(Empty()){
        std::cout << "the queue is empty\n";
        return T();
    }
    // return the back
    return q_array[q_back];
}

// clear
template <typename T>
void queueArr<T>::Clear(){
    // reset the q_size, q_front, q_back
    q_size = 0;
    q_front = 0;
    q_back = -1;
}
```

```cpp
template <typename T>
void queueArr<T>::Enqueue(T value){
    // check if the queue is full
    if(Full()){
        std::cout << "Queue is full\n";
        return;
    }

    // move q_back circularly
    q_back = (q_back + 1) % q_capacity;

    // store the value to the back
    q_array[q_back] = value;

    // increment the q_size
    q_size++;
}
```

```
168  // T Dequeue()
169  template <typename T>
170  T queueArr<T>::Dequeue(){
171      // check if empty
172      if(Empty()){
173          std::cout << "Queue is empty\n";
174          return T();
175      }
176
177      // create a temp variable to store current front
178      T temp = q_array[q_front];
179
180      // move q_front circularly
181      q_front = (q_front + 1) % q_capacity;
182
183      // decrement q_size
184      q_size--;
185
186      // reset indexes if queue becomes empty
187      if(Empty()) Clear();
188
189      // return temp variable
190      return temp;
191  }
192
193  #endif
```

Figure 2. Header File of the Array Implementation of Queues.

This header file contains the array implementation of queues, which differs from the linked list implementation with the increased number of functions and instructions, and a class containing private and public variables. The array class contains the declaration of the pointers and variables, and the core elements of the program, which are the constructor, copy constructor, destructor, and the queue operations. The constructor sets up the variables and the array that will be used by the other functions to store the values properly. The copy constructor copies the values of the other variable to its own set of variables. The copy assignment operator takes another copy of the values and creates a new array and set of variables. Since this is an array implementation, each operator sets its own variables.

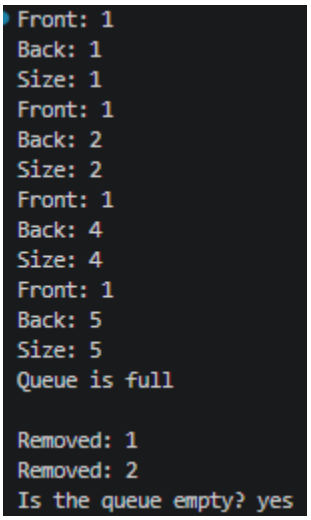


Figure 3. Console Output of the Array Implementation of Queues.

The console output displays the queue operations performed in the driver code, which contains the values of 1, 2, 3, 4, and 5 similar to the previous output before. The difference is the additional display of the queue size of the total number of elements in the queue. To allow for the queue size to match, there is a plus one in the front and back variables in the enqueue and dequeue functions. When reaching the maximum value of 5, the same enqueue function displays a message telling that the queue is full. The clear function then removes all the elements and the empty function checks if the queue is empty.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

ILO C

```cpp
#ifndef PRINTERARR_H
#define PRINTERARR_H
#include <iostream>
#include <string>
// job class
class Job{
    public:
        int id;
        std::string user;
        int pages;
};
// printer class
class Printer{
    private:
        // variables
        Job* q_array;
        size_t q_capacity;
        size_t q_size;
        int q_front;
        int q_back;

    public:
        // constructor
        Printer(size_t capacity = 10){
            q_capacity = capacity;
            q_size = 0;
            q_front = 0;
            q_back = -1;
            q_array = new Job[q_capacity];
        }
        // destructor
        ~Printer(){
            delete[] q_array;
        }
        // checks
        bool Empty(){
            return q_size == 0;
        }

        bool Full(){
            return q_size == q_capacity;
        }
```

```cpp
        // job operations
        void AddJob(Job newJob){
            if(Full()){
                std::cout << "Printer queue is full\n";
                return;
            }
            // using circular method
            q_back = (q_back + 1) % q_capacity;
            q_array[q_back] = newJob;
            q_size++;

            std::cout << "Print job added\n";
        }

        Job RemoveJob(){
            if(Empty()){
                std::cout << "No print jobs\n";
                return Job();
            }
            // using circular method
            Job temp = q_array[q_front];
            q_front = (q_front + 1) % q_capacity;
            q_size--;
            // clear
            if(Empty()){
                Clear();
            }

            return temp;
        }
```

```
74          void ProcessJobs(){
75              if(Empty()){
76                  std::cout << "There are no print jobs to process\n";
77                  return;
78              }
79              // process all jobs
80              while(!Empty()){
81                  Job temp = RemoveJob();
82                  std::cout << "Printing Job ID: " << temp.id << "   User: " <<
                    temp.user << "   Pages: " << temp.pages << std::endl;
83              }
84              // message
85              std::cout << "Print Jobs DONE\n";
86          }
87
88          void ShowJobs(){
89              if(Empty()){
90                  std::cout << "There are no pending print jobs\n";
91                  return;
92              }
93              int index = q_front;
94              std::cout << "\nPending Print Jobs\n";
95              // display jobs
96              for(size_t i = 0; i < q_size; i++){
97                  std::cout << "Job ID: " << q_array[index].id << "   User: " <<
                    q_array[index].user << "   Pages: " << q_array[index].pages <<
                    std::endl;
98
99                  index = (index + 1) % q_capacity;
100             }
101         }
102         // clear, reset
103         void Clear(){
104             q_size = 0;
105             q_front = 0;
106             q_back = -1;
107         }
108 };
109 #endif
```

Figure 1. Header File of the Printer Queue Program Using Array Implementation.

This header file contains the array implementation of the printer queue program, which contains the job and printer classes for storing the variables and performing the queue operations. Since this program is based on a circular array implementation of queues, this includes the basic constructor and destructor, and also the full or empty functions, and the add, remove, process, and show functions for the printer job. Similar to the array-based queue header file, this uses the circular method of adding and clearing the list, which contains a variable added by one and placed in a modulus against the value of the capacity.

```cpp
#include <iostream>
#include "printerArr.h"

int main(){
    Printer printer(10);
    int choice;
    // program loop
    do{
        std::cout << "\n===== PRINTERS =====\n";
        std::cout << "1. Add Print Job\n";
        std::cout << "2. Process All\n";
        std::cout << "3. Remove Print Job\n";
        std::cout << "4. Show Print Jobs\n";
        std::cout << "5. Clear All\n";
        std::cout << "6. Exit\n";
        std::cout << "Choice: ";
        std::cin >> choice;
        // if else if...
        if(choice == 1){
            Job temp;

            std::cout << "Job ID: ";
            std::cin >> temp.id;
            std::cout << "User Name: ";
            std::cin >> temp.user;
            std::cout << "Pages: ";
            std::cin >> temp.pages;

            printer.AddJob(temp);
        }

        else if(choice == 2){
            printer.ProcessJobs();
        }

        else if(choice == 3){
            Job temp = printer.RemoveJob();
            if(temp.id != 0){
                std::cout << "Removed Job\n";
                std::cout << "Job ID: " << temp.id << std::endl;
                std::cout << "User: " << temp.user << std::endl;
                std::cout << "Pages: " << temp.pages << std::endl;
            }
        }

        else if(choice == 4){
            printer.ShowJobs();
        }

        else if(choice == 5){
            printer.Clear();
            std::cout << "All print jobs cleared.\n";
        }

    } while(choice != 6);

    return 0;
}
```

Figure 2. Driver Code of the Printer Queue Program Using Array Implementation.

This driver code contains the menu system for the printer queue program, which contains the choices to call the functions in the header file to perform the queue operations. This menu displays the choices for user input, and contains a series of if and else if statements for the choices selected in the input menu. To print the values, the program accesses the members of the function and class.

```
===== PRINTERS =====
1. Add Print Job
2. Process All
3. Remove Print Job
4. Show Print Jobs
5. Clear All
6. Exit
Choice: 1
Job ID: 123
User Name: ABC
Pages: 11
Print job added
```

```
===== PRINTERS =====
1. Add Print Job
2. Process All
3. Remove Print Job
4. Show Print Jobs
5. Clear All
6. Exit
Choice: 4

Pending Print Jobs
Job ID: 123   User: ABC   Pages: 11
Job ID: 456   User: DEF   Pages: 22
Job ID: 789   User: GHI   Pages: 33
```

```
===== PRINTERS =====
1. Add Print Job
2. Process All
3. Remove Print Job
4. Show Print Jobs
5. Clear All
6. Exit
Choice: 3
Removed Job
Job ID: 123
User: ABC
Pages: 11
```

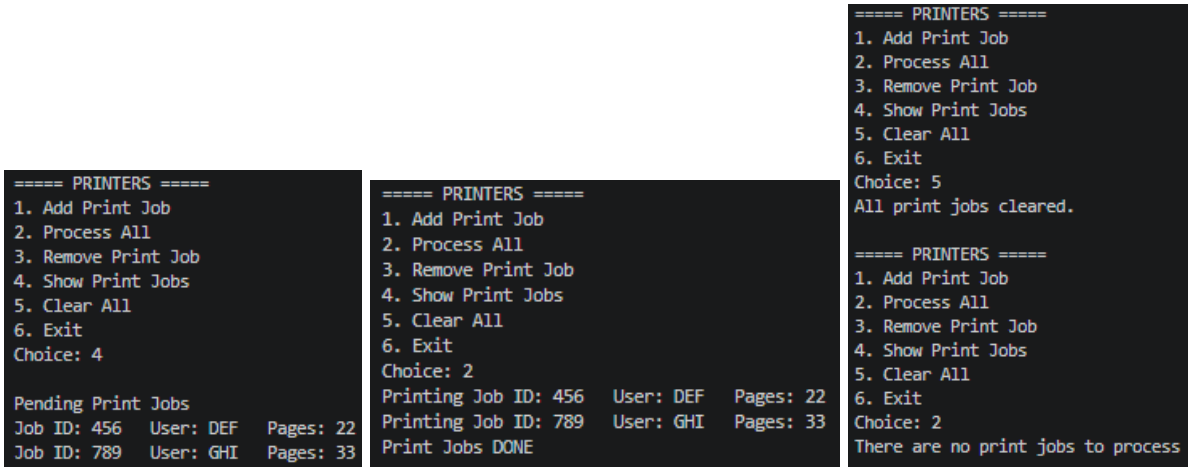


Figure 3. Output of the Printer Queue Program Using Array Implementation.

This console output of the program contains the user input menu and the printer job operations, such as add job (enqueue), remove job (dequeue), show print jobs (displayALL), and clear all. This is an application of the queue algorithm using the array implementation modified to function as a print job system to manage print queues, perform print jobs, and list down the added print jobs. The displays contain the printing job ID, user name, and number of pages, as expected in a printer queue program. When the queue is empty or all the indexes are removed and set to zero, the empty function checks this to display the message telling that the queue is empty.

Why did you use arrays or linked lists?

Using the array implementation of queues, specifically the circular version, allows for a generally safer method for implementing a printer program since the maximum number of the print job queue can be set in the source code. Because print jobs follow the FIFO method, using the array method can use memory more efficiently with a faster random access and smaller memory usage. Arrays in memory are also easier to visualize since it behaves like a table of values and addresses, while linked lists are more abstract since this method involves nodes connected to one another in a linear manner.

CONCLUSION

Concluding this hands-on activity, the ILOs have successfully demonstrated the concepts of queues in various implementations, such as using the STL library, header files, and two different methods shown in the activity, which are linked lists and circular arrays. The supplementary activities encourage the practice of writing custom-made header files based on the given algorithms from the procedures to function as the backbone of the printer program, which becomes more essential for reusable code to be implemented in other activities. The activities show the notable similarities of print jobs to queue operations, such as the enqueue, dequeue, and the empty and full check functions. By using these algorithms in actual scenarios, it is possible to write programs that solve problems in a way that is clean, structured, and minimal in bugs and errors.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.